Core Classes

Book.cs

```
namespace LibraryManagementSystem
{
    public class Book
    {
        public string Title { get; set; }
        public string Author { get; set; }
        public string ISBN { get; set; }
        public bool IsBorrowed { get; private set; }

        public Book(string title, string author, string isbn)
        {
            Title = title;
            Author = author;
            ISBN = isbn;
            IsBorrowed = false;
        }

        public void Borrow()
        {
            if (IsBorrowed)
                throw new InvalidOperationException("Book is already borrowed.");

            IsBorrowed = true;
        }

        public void Return()
        {
            IsBorrowed = false;
```

```
        }
    }
}
```

Borrower.cs

```
using System.Collections.Generic;

namespace LibraryManagementSystem
{
    public class Borrower
    {
        public string Name { get; set; }
        public string LibraryCardNumber { get; set; }
        public List<Book> BorrowedBooks { get; set; }

        public Borrower(string name, string cardNumber)
        {
            Name = name;
            LibraryCardNumber = cardNumber;
            BorrowedBooks = new List<Book>();
        }

        public void BorrowBook(Book book)
        {
            book.Borrow();
            BorrowedBooks.Add(book);
        }

        public void ReturnBook(Book book)
        {
            book.Return();
            BorrowedBooks.Remove(book);
```

```
        }
    }
}
```

Library.cs

```
using System;
using System.Collections.Generic;
using System.Linq;

namespace LibraryManagementSystem
{
    public class Library
    {
        public List<Book> Books { get; set; } = new List<Book>();
        public List<Borrower> Borrowers { get; set; } = new List<Borrower>();

        public void AddBook(Book book)
        {
            Books.Add(book);
        }

        public void RegisterBorrower(Borrower borrower)
        {
            Borrowers.Add(borrower);
        }

        public void BorrowBook(string isbn, string cardNumber)
        {
            var book = Books.FirstOrDefault(b => b.ISBN == isbn);
            var borrower = Borrowers.FirstOrDefault(b => b.LibraryCardNumber == cardNumber);

            if (book == null)
```

```
            throw new Exception("Book not found");

        if (borrower == null)
            throw new Exception("Borrower not found");

        borrower.BorrowBook(book);
    }

    public void ReturnBook(string isbn, string cardNumber)
    {
        var book = Books.FirstOrDefault(b => b.ISBN == isbn);
        var borrower = Borrowers.FirstOrDefault(b => b.LibraryCardNumber == cardNumber);

        if (book == null || borrower == null)
            throw new Exception("Invalid return request");

        borrower.ReturnBook(book);
    }

    public List<Book> ViewBooks()
    {
        return Books;
    }

    public List<Borrower> ViewBorrowers()
    {
        return Borrowers;
    }
  }
}
```

Task 2: Unit Tests (MSTest)

LibraryTests.cs

```
using Microsoft.VisualStudio.TestTools.UnitTesting;
using LibraryManagementSystem;
using System.Linq;

namespace LibraryTests
{
    [TestClass]
    public class LibraryTests
    {
        private Library _library;

        [TestInitialize]
        public void Setup()
        {
            _library = new Library();
        }

        // ✅ Add Book
        [TestMethod]
        public void AddBook_ShouldAddBookToLibrary()
        {
            var book = new Book("Test Book", "Author", "123");

            _library.AddBook(book);

            Assert.AreEqual(1, _library.Books.Count);
        }

        // ✅ Register Borrower
```

```
[TestMethod]
public void RegisterBorrower_ShouldAddBorrower()
{
    var borrower = new Borrower("John", "C1");

    _library.RegisterBorrower(borrower);

    Assert.AreEqual(1, _library.Borrowers.Count);
}

// ✅ Borrow Book
[TestMethod]
public void BorrowBook_ShouldMarkBookAsBorrowed()
{
    var book = new Book("Test", "Author", "123");
    var borrower = new Borrower("John", "C1");

    _library.AddBook(book);
    _library.RegisterBorrower(borrower);

    _library.BorrowBook("123", "C1");

    Assert.IsTrue(book.IsBorrowed);
    Assert.AreEqual(1, borrower.BorrowedBooks.Count);
}

// ✅ Return Book
[TestMethod]
public void ReturnBook_ShouldMakeBookAvailable()
{
    var book = new Book("Test", "Author", "123");
```

```
    var borrower = new Borrower("John", "C1");

    _library.AddBook(book);
    _library.RegisterBorrower(borrower);

    _library.BorrowBook("123", "C1");
    _library.ReturnBook("123", "C1");

    Assert.IsFalse(book.IsBorrowed);
    Assert.AreEqual(0, borrower.BorrowedBooks.Count);
}

// ✅ View Books
[TestMethod]
public void ViewBooks_ShouldReturnAllBooks()
{
    _library.AddBook(new Book("B1", "A1", "1"));
    _library.AddBook(new Book("B2", "A2", "2"));

    var books = _library.ViewBooks();

    Assert.AreEqual(2, books.Count);
}

// ✅ View Borrowers
[TestMethod]
public void ViewBorrowers_ShouldReturnAllBorrowers()
{
    _library.RegisterBorrower(new Borrower("John", "C1"));
    _library.RegisterBorrower(new Borrower("Jane", "C2"));
```

```
            var borrowers = _library.ViewBorrowers();


            Assert.AreEqual(2, borrowers.Count);
        }
    }
}
```